

cs-mail Rust Reference Architecture

C-SQD

Draft 0.6 — August 2026

Abstract

This document explains a Rust architecture for the *cs-mail* protocol by following one recipient acceptance from arrival through final settlement. The central operation is a pure transition: authenticated intent and a complete state snapshot become a settlement manifest. One atomic transaction then applies the relationship change, settles every eligible bond, releases persistence reserves, preserves principal-level repeated-attempt state, records balanced ledger entries, and schedules external effects. The architecture also gives reserved but unadmitted attempts a short cancellation window with full release. The same execution model provides the organizing principle for Rust types, crate boundaries, persistence, concurrency, privacy, encryption, testing, and eventual federation.

Reading guide. The protocol specification defines what every conforming implementation must do. This document explains one way to preserve those semantics in Rust. It begins with a complete acceptance, names the architectural vocabulary, then moves from the pure kernel to durable execution, crate boundaries, privacy domains, federation, and testing. Code fragments are illustrative rather than a wire or storage API commitment.

1 One Acceptance from Beginning to End

Alice has not yet been accepted by Bob. She has made three contact attempts, each with a valid bond. All three bonds are still within their decision windows. Bob opens one of the messages and chooses to accept Alice.

The first admitted attempt opened one relationship solicitation for Alice’s visible identity. The second and third attempts joined that solicitation. They may have updated its grouped contents or unread count, but they did not produce two more requests asking Bob to establish the same relationship.

Although the interface places the action beside one message, Bob is accepting a communicative relationship. The system must establish permission from Alice’s visible protocol identity I_A to Bob’s visible protocol identity I_B . It must also refund $C + S$ for all three decision-open admitted bonds, release every active persistence reserve associated with attempts from I_A to I_B , preserve the repeated-attempt state shared by Alice’s private principal and Bob’s identity, and cancel the three scheduled expiries. Acceptance of I_A does not forgive attempts made through another public identity controlled by Alice.

Those effects form one protocol outcome. If permission becomes visible while one bond remains available for later rejection, the implementation has contradicted the protocol. If the money is refunded but permission remains unknown, it has done the same. This relationship-wide outcome is the architectural center of the Rust implementation.

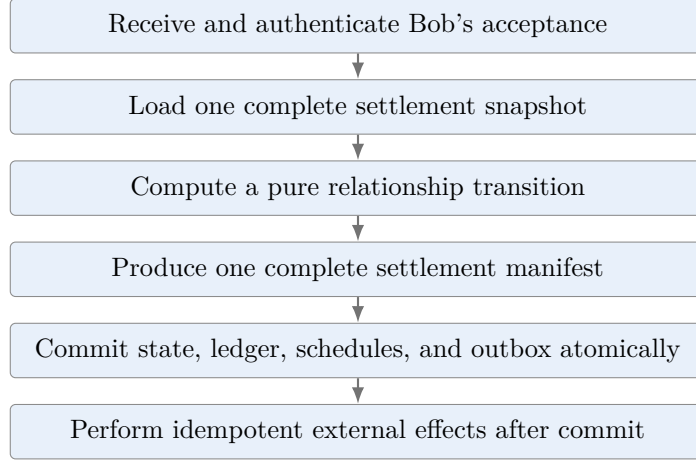


Figure 1: The execution path of one relationship decision.

1.1 The Acceptance Arrives

Bob’s client sends an `AcceptRelationship` command. The command names the directed relationship, carries Bob’s expected relationship version and an idempotency key, and is signed by an authorized operational key. The ingress service verifies that signature and durably records when the command arrived.

That receipt time is authoritative. Bob’s device clock may be wrong, so the client timestamp cannot decide whether a bond beat its deadline. The ingress service returns a signed receipt and passes an authenticated command onward. From this point, the implementation works with a fact—Bob authorized this exact command at canonical time t_a —rather than a boolean claiming that someone authenticated something.

1.2 The Application Loads One Complete View

The application begins a protected database transaction and loads the facts that the acceptance can change. These are the current relationship, every nonterminal bond for (I_A, I_B) , the repeated-attempt state for Alice’s private principal P_A and Bob’s identity I_B , every active persistence reserve for (P_A, I_B) with its originating public-identity reference, and the scoped ledger ownership references needed for settlement.

Canonical state and time partition the bonds. A reserved bond is unadmitted and must be cancelled with a full $C + S + L_k$ return. An admitted bond with $t_a \leq$ decision deadline is decision-open and must have $C + S$ refunded by this acceptance. An admitted bond whose deadline precedes t_a is logically expired and cannot be reopened, even when its background expiry worker has not yet written the final ledger entries.

Completeness matters financially. Loading the first hundred bonds while silently omitting the hundred-and-first would not be an acceptable performance compromise. It would under-refund Alice. The storage layer must prove that the snapshot contains the complete nonterminal set for the relationship at the transaction boundary.

1.3 The Protocol Computes the Meaning

The application passes the authenticated command and complete snapshot to a pure, synchronous Rust function. The function reads no database, asks no service for the time, sends no message, and

moves no money. It validates the command against the snapshot and computes what the entire relationship decision means.

Its result is a *transition manifest*. For this acceptance, the manifest says that the relationship becomes accepted; all three decision-open bonds become terminally accepted; each bond's $C + S$ returns to Alice; every active persistence reserve associated with attempts from I_A to I_B is released; principal-level repeated-attempt state is preserved; and the scheduled expiries are cancelled. If the snapshot also contained a reserved but unadmitted attempt for the pair, the manifest would cancel it and return its entire $C + S + L_k$. It also contains the balanced journal postings, authenticated protocol events, and post-commit work implied by those changes.

The application does not receive a small result such as `RelationshipState::Accepted` and then independently rediscover all the financial consequences. If it did, acceptance semantics would be duplicated across controllers, workers, and storage code. The manifest is complete precisely so that infrastructure cannot accidentally invent a second interpretation.

1.4 One Transaction Commits the Outcome

Before applying the manifest, storage verifies its preconditions. The relationship, bonds, reserves, attempt state, quote versions, and completeness boundary must be the same versions used by the pure transition. A concurrent reservation or recipient decision changes one of those versions and causes the entire command to reload rather than allowing a partial commit.

The transaction then appends the balanced ledger batch, updates protocol state, records authenticated events, changes scheduled work, and inserts transactional outbox records. It commits all of them together. Until that commit succeeds, no other provider is told that Alice is accepted and no notification is sent.

After commit, workers deliver receipts, notifications, and any federated effects described by the outbox. Those workers may retry. Stable idempotency keys ensure that a crash after sending but before acknowledgement cannot repeat a settlement or create a second external effect.

The full execution can now be summarized in one sentence: an authenticated relationship command and a complete snapshot produce one deterministic manifest, which infrastructure commits atomically before performing retryable external work. The rest of the architecture makes that sentence precise.

2 The Small Vocabulary

Six ideas recur throughout the implementation. Their distinctions are more important than their names.

2.1 A Command Expresses Intent

A command says what an actor is asking the protocol to do. Bob asks to accept a relationship. A sender asks to reserve or admit an attempt. A scheduled worker asks to expire a bond. A command is not permission to mutate a row directly; it must be authenticated, validated against current state, and interpreted by the transition logic.

2.2 A Snapshot Describes Prior Facts

A snapshot is the complete versioned state required to evaluate one command. It is not an arbitrary cache of convenient objects. Its completeness and transaction boundary are part of correctness. The

same command evaluated against a different relationship version is a different protocol situation.

2.3 A Transition Computes Meaning

A transition is the pure calculation that combines intent and prior facts. It either returns a protocol error or a complete proposed outcome. Because time, policy, authentication evidence, and prior state are explicit inputs, identical inputs produce identical results.

2.4 A Manifest Describes the Whole Proposed Outcome

A manifest collects every change that must appear atomically: domain updates, ledger postings, schedule changes, events, and post-commit intentions. It is still data. Creating a manifest does not mean any effect has happened.

2.5 A Commit Makes the Outcome Durable

The commit checks that the manifest's preconditions remain true and then makes all of its durable consequences visible together. The commit boundary is where a proposed transition becomes canonical protocol history.

2.6 An Effect Happens After Commit

An effect communicates with the outside world: delivering content, sending a notification, presenting a receipt, or contacting another provider. Effects are driven from a transactional outbox and are idempotent because the application may need to repeat them after a crash or timeout.

This vocabulary prevents a common architectural confusion. An event records what the committed protocol says happened. An outbox intent asks a worker to do something because it happened. The two may share a cause identifier, but they are not interchangeable.

3 The Pure Rust Kernel

The protocol kernel is a synchronous library whose public center is approximately the following function:

```
pub fn transition(
    snapshot: &SettlementSnapshot,
    command: Authorized<ProtocolCommand>,
    context: &TransitionContext,
) -> Result<TransitionManifest, ProtocolError> {
    // Validate state and derive the complete atomic consequence.
    // Perform no I/O here.
}
```

Synchronous does not mean that the surrounding server is synchronous. It means this function performs finite CPU and memory work without yielding to a clock, database, queue, or network. An asynchronous application can call it inside an ordinary transaction workflow.

Purity makes the protocol easier to reason about. A test can place acceptance one nanosecond before a deadline without mocking a global clock. A property test can replay the same sequence and require the same manifest. A federated implementation in another language can compare its result with a Rust reference vector.

3.1 Authenticated Commands Are Values

Wire bytes pass through three gates before reaching the kernel. A versioned decoder first checks structural validity and size limits. The security layer then verifies the signature, signing domain, key status, and canonical receipt. Finally, a conversion validates the domain form: identifier scope, amount representation, expected versions, and command-specific fields.

The result can be represented by a wrapper whose constructor is unavailable to ordinary application code:

```
pub struct Authorized<T> {  
    command: T,  
    actor: ActorRef,  
    key: OperationalKeyRef,  
    command_digest: CommandDigest,  
    receipt: CanonicalReceipt,  
}
```

The wrapper binds a particular command digest to a verified actor and receipt. A field such as `is_authenticated: true` would not provide the same safety; any caller able to construct the value could assert it.

3.2 Context Makes Hidden Inputs Visible

The transition context contains values that are authoritative for this evaluation but are not persistent relationship objects:

```
pub struct TransitionContext {  
    pub now: CanonicalTime,  
    pub journal_position: ProposedJournalPosition,  
    pub protocol_version: ProtocolVersion,  
    pub policy: PolicySnapshot,  
}
```

The policy snapshot is immutable for the transition. The kernel does not query a live configuration service halfway through acceptance. Each bond's quoted C , S , L_k , admission-window duration, decision-window duration, and release rule remain fixed in its validated terms. Reservation materializes the admission deadline; admission later materializes the decision deadline.

3.3 State Is Explicit and Exhaustive

Relationship and bond states fit naturally into Rust enums:

```
pub enum RelationshipState {
    Unknown,
    Accepted,
    Rejected,
    Revoked,
    Blocked,
}

pub enum BondState {
    Reserved,
    Admitted,
    Accepted { event: EventRef },
    Rejected { event: EventRef },
    Expired { event: EventRef },
    CancelledUnadmitted { event: EventRef },
}
```

Exhaustive matching forces a new protocol version to confront every transition site. The fields should remain private to the protocol module so that callers can inspect a terminal state but cannot assign it without producing the required ledger and event consequences.

Rejected and **Blocked** deliberately encode different recipient intent. Rejection settles the active solicitation but allows a later eligible bonded attempt after backoff. Blocking settles current decision-open admitted bonds under the same allocation, fully cancels unadmitted reservations, and prevents terms, reservation, optional capability use, and admission until an authenticated unblock returns the pair to **Rejected**. It does not silently accept the relationship or reset attempt state.

A separate Rust type for every state can make a local object lifecycle elegant, but it becomes cumbersome when a relationship decision hydrates and settles a heterogeneous collection of bonds. A private enum combined with centralized transition authority preserves clarity without turning storage reconstruction into a web of typestate conversions.

3.4 The Manifest Is Concrete

The transition result gathers the whole proposed outcome:

```
pub struct TransitionManifest {
    pub preconditions: Preconditions,
    pub relationship_update: Option<RelationshipUpdate>,
    pub bond_updates: Vec<BondUpdate>,
    pub attempt_update: Option<AttemptUpdate>,
    pub reserve_updates: Vec<ReserveUpdate>,
    pub solicitation_update: Option<SolicitationUpdate>,
    pub ledger_batch: LedgerBatch,
    pub schedule_changes: Vec<ScheduleChange>,
    pub protocol_events: Vec<ProtocolEvent>,
    pub outbox_intents: Vec<EffectIntent>,
}
```

Private constructors validate relationships among these fields. An acceptance manifest must represent every decision-open admitted bond, exactly one $C + S$ refund for each, full cancellation of every reserved but unadmitted attempt, exactly one release for each active persistence reserve associated with the accepted public identity, preservation of principal-level attempt state and other identities' reserves, a balanced ledger batch, an accepted relationship version, closure of the active solicitation, and cancellation of the appropriate timeout and expiry operations. It cannot be assembled incrementally by unrelated handlers.

4 Types That Carry Protocol Meaning

Rust's type system is useful when it expresses distinctions already present in the protocol. It should prevent category errors without trying to encode the entire state machine as an unreadable type exercise.

4.1 Identifiers Belong to Privacy Domains

An address shown to a user, a private principal, a relationship handle, a ledger bond reference, and a content reference all identify something. They are not interchangeable identifiers.

```
pub struct ProtocolIdentity(String);
pub struct PrincipalRef([u8; 32]);
pub struct RelationshipHandle([u8; 32]);
pub struct AttemptPairRef([u8; 32]);
pub struct LedgerBondRef([u8; 32]);
pub struct ContentRef([u8; 32]);
```

The real fields should be private and construction should validate length, scope, and domain. Sensitive identifiers should implement a redacted `Debug`, or no `Debug` at all. An error logged with `{:?}` must not expose a principal or cross-domain mapping.

A relationship service handle need not be the reference used by the ledger or telemetry. Requiring explicit translation at an authorized boundary is safer than passing one universal event or user identifier through every subsystem.

4.2 Money Is Checked Integer Arithmetic

Protocol money uses an integer number of the smallest supported unit together with an explicit settlement unit:

```
pub struct MinorUnits(u64);

pub enum SettlementUnit {
    UsdCent,
    // New units require an explicit protocol version.
}

pub struct Money {
    units: MinorUnits,
    settlement_unit: SettlementUnit,
}
```

The equation $R_k = C + S + L_k$ becomes checked addition that rejects overflow and mismatched settlement units. A validated quote stores the proven total rather than asking every caller to reproduce the arithmetic. Floating-point amounts have no place in protocol settlement.

4.3 Different Clocks Have Different Types

A device’s claimed timestamp and the canonical ingress time have different authority and should not share a Rust type:

```
pub struct ClientTimestamp(i64);
pub struct CanonicalTime(i64);
pub struct AdmissionDeadline(CanonicalTime);
pub struct DecisionDeadline(CanonicalTime);
pub struct ScheduledReleaseAt(CanonicalTime);
```

The compiler can then prevent accidental comparison of a client timestamp with a financial deadline. Conversion is possible only at a boundary that explicitly defines what the conversion means.

4.4 Versions Also Describe Different Things

Relationship state, repeated-attempt state, wire formats, protocol semantics, events, policies, and database schemas all have versions. One generic integer called `version` obscures which compatibility or concurrency rule is being checked. Small newtypes such as `RelationshipVersion`, `PolicyVersion`, and `WireVersion` make the distinction visible.

5 Durable Execution

The pure kernel knows what should happen. The application and storage layers make that outcome durable without changing its meaning.

5.1 Reservation Is a Short-Lived Pre-Admission Hold

`ReserveAttempt` moves $C + S + L_k$ from available to reserved ownership and records an admission deadline. It does not create a recipient solicitation, advance repeated-attempt state, or start the recipient’s decision window. The same transaction schedules an idempotent `CancelReservedAttempt` command.

`AdmitAttempt` may commit only at or before that deadline and only when the encrypted content and a retryable delivery intent become durable in the same transaction. It records canonical admission time, derives the decision deadline from the quoted duration, replaces the cancellation schedule with an expiry schedule, advances repeated-attempt state, and opens or joins the solicitation. After that commit, an outbox retry is delivery recovery rather than grounds for rolling back the protocol attempt.

Sender cancellation, admission timeout, or verified pre-admission technical failure all produce the same terminal result: `CancelledUnadmitted`. The full $C + S + L_k$ returns, no decision deadline or solicitation exists, and attempt state does not advance. Admission and cancellation serialize on the reserved bond version, so neither can commit after the other.

5.2 A Snapshot Carries Completeness Evidence

The settlement snapshot contains all nonterminal bonds rather than only the bonds that a convenient page of results happened to return:

```
pub struct SettlementSnapshot {
    pub relationship: Versioned<Relationship>,
    pub nonterminal_bonds: CompleteSet<Versioned<Bond>>,
    pub attempt_state: Versioned<RepeatedAttemptState>,
    pub active_reserves: CompleteSet<Versioned<PersistenceReserve>>,
    pub ownership: SettlementOwnership,
}
```

`CompleteSet` is not a decorative wrapper around `Vec`. The storage adapter constructs it only after reading from an index and transaction boundary that prove completeness for the directed pair. The protocol kernel can refuse a partial collection whose completeness has not been established.

The snapshot includes every nonterminal bond because state and canonical time must divide it into reserved-but-unadmitted, decision-open admitted, and logically expired admitted bonds. A relationship decision fully cancels the first group, settles the second, and cannot reopen the third.

A practical partitioning starting point. Relationship state and its nonterminal-bond index can be partitioned by recipient protocol identity and directed relationship handle. Repeated-attempt state and active persistence-reserve indexes can be partitioned by the recipient-scoped (P_i, I_j) assertion, but each reserve also retains its originating public-identity relationship so acceptance can release only the matching subset. Ledger accounts and reservations can be partitioned by ledger ownership. This layout is guidance, not protocol meaning. The storage adapter must still prove that a relationship decision sees the complete nonterminal-bond set and the complete identity-filterable active-reserve set at one protected boundary. Pagination, sharding, or an asynchronously updated index cannot excuse partial settlement. Co-location may make the initial transaction simpler; a later distributed design must preserve the same manifest and atomic visibility.

5.3 The Transaction Applies a Manifest, Not Business Logic

The application workflow is intentionally mechanical:

```
store.transaction(|tx| {
  let snapshot = tx.load_settlement_snapshot(&scope)?;
  let manifest = protocol::transition(
    &snapshot,
    authorized_command,
    &context,
  )?;

  tx.verify(&manifest.preconditions)?;
  tx.ledger().append(&manifest.ledger_batch)?;
  tx.apply_protocol_updates(&manifest)?;
  tx.enqueue_effects(&manifest.outbox_intents)?;
  tx.apply_schedule_changes(&manifest.schedule_changes)?;
  tx.commit()
})
```

The transaction does not contain another switch statement explaining what acceptance means. It verifies and applies the complete result returned by the kernel. In the initial deployment, a serializable database transaction or an equivalently protected version-and-lock strategy is the simplest correct choice.

The relevant rows include the relationship, its nonterminal-bond index, the principal–recipient attempt state, active reserves, and affected ledger accounts. A later distributed design may replace this local transaction with a prepared manifest and distributed commit, but it must preserve the same atomic result.

5.4 The Ledger Is Authoritative for Money

The ledger journal is append-only and double-entry. Available and reserved balances may be fast materialized projections, but the journal remains the authoritative account of ownership changes.

```
pub struct LedgerBatch {
  pub batch_id: LedgerBatchId,
  pub cause: SettlementCause,
  pub postings: Vec<Posting>,
}

pub struct Posting {
  pub account: LedgerAccountRef,
  pub side: DebitOrCredit,
  pub money: Money,
}
```

The ledger verifies that postings balance within each settlement unit, reserved value exists before it is released or transferred, an idempotency key identifies one payload, and a terminal bond cause has not already been posted. The protocol chooses the settlement meaning; the ledger proves its accounting structure.

A terminal protocol event is never rewritten. If an external funding reversal or later adjudication requires an accounting correction, a separately authorized compensating batch records that new

fact. It does not pretend the original acceptance, rejection, or expiry never occurred.

Ledger records use scoped account and bond references. They do not contain email addresses, subjects, content references shared with content storage, or private principal identifiers.

5.5 The Outbox Separates Commit from Communication

Notifications, delivery requests, federated messages, and settlement receipts are external effects. Performing one before the database commit can expose a state the protocol later fails to record. The transaction therefore stores an **EffectIntent** in the outbox alongside the state and ledger changes.

An asynchronous worker reads committed intents and performs them. Each intent has a stable idempotency key derived from its committed protocol cause. A receiver returns the original result for an exact replay and rejects a different payload using the same key.

The same model applies to scheduled work. Unadmitted-reservation cancellation, admitted-bond expiry, deferred reserve release, and retention cleanup are represented by records containing the target, earliest execution time, expected version, and idempotency key. A worker wakes and submits an ordinary command against current state. It never assumes that state remained unchanged because a job was scheduled earlier.

No permanently running actor or Rust task is needed for every possible sender–recipient pair. Relationship state remains passive data. Queues distribute finite work, and indexes locate the state required for each command.

5.6 Relationship Solicitation Is a Coalesced Effect

The first admission for an unaccepted directed relationship opens a solicitation generation. The transition manifest records that state change and one outbox intent asking the recipient surface to present a relationship-establishment request. Later admissions update the active solicitation projection but produce no second establishment intent:

```
pub struct SolicitationProjection {
    pub relationship: RelationshipHandle,
    pub generation: SolicitationGeneration,
    pub status: SolicitationStatus,
    pub admitted_attempts: CompleteSet<AttemptRef>,
    pub notification_emitted: bool,
    pub version: SolicitationVersion,
}
```

The storage transaction serializes admissions on the relationship and solicitation version, or enforces an equivalent unique open-generation key. The establishment effect uses an idempotency key derived from the relationship handle, generation, and effect type. A crash, replay, or concurrent second admission can therefore neither open a duplicate generation nor emit another establishment request.

The establishment effect contains only scoped relationship and presentation references. It does not carry a plaintext subject, content preview, legal identity, balance, or unrelated relationship history. Recipient settings may choose a quiet inbox presentation instead of an interruptive push without allowing later attempts to manufacture repeated establishment requests.

This is effect coalescing, not attempt idempotency. Every admitted message retains its own content reference, bond, decision deadline, persistence reserve, and attempt-level consequence. A recipient surface may quietly update the grouped messages or unread count. Dismissing that surface is not an authenticated relationship decision and changes no settlement.

Acceptance, rejection, and blocking close the generation in the same manifest as the relationship and financial outcome. Rejection allows a later eligible attempt after backoff; blocking prevents one until the recipient unblocks. Expiry closes the generation only after the complete episode projection shows that no associated admitted bond remains decision-open. A later eligible attempt may open the next generation and one new establishment effect.

5.7 Acceptance and Expiry Share One Ordering Rule

An expiry worker is not the authority on whether Bob acted in time. It is the mechanical materializer of an outcome implied by canonical time and the decision journal. Before expiry becomes terminal, the worker must re-read current bond state and check for a decision durably received by the deadline.

If Bob's acceptance was received on or before a bond's deadline, acceptance wins even when an expiry worker began first. If acceptance arrived afterward, the old bond expires, but the relationship can still become accepted prospectively and newer decision-open admitted bonds can still be refunded. Expiry of the older bond must not undo the later accepted relationship.

Concurrent reservation follows the relationship version boundary. A reservation that commits before acceptance belongs to the complete nonterminal set. If still unadmitted, acceptance cancels it and returns $C + S + L_k$; if admission also committed first, acceptance settles it only when decision-open. A reservation ordered afterward sees an accepted relationship, creates no bond, and releases any provisional hold.

5.8 Selective Event Sourcing Is Enough

Explicit protocol events are valuable, but every current value need not be rebuilt by replaying a global history. Relationship and repeated-attempt state can be stored as versioned current records with append-only authenticated audit streams. The ledger journal remains authoritative for money. Snapshots and projections can be rebuilt within bounded partitions.

Storage interfaces should describe these real operations. A generic `Repository<T>` often hides the difference between loading one relationship, proving a complete nonterminal-bond set, and appending a balanced ledger batch. Purpose-specific interfaces make their transaction guarantees reviewable.

5.9 Errors Preserve Uncertainty Honestly

A protocol error such as `QuoteExpired`, `BackoffActive`, `VersionConflict`, or `AlreadyTerminal` is a deterministic result of command and state. An infrastructure error says the application could not obtain or commit that result because storage, a queue, or a remote provider was unavailable.

An unknown commit outcome must not be converted into a new command automatically. The application first queries the original idempotency key. If the transaction committed, it returns the recorded result. If it provably did not, the identical payload may be retried.

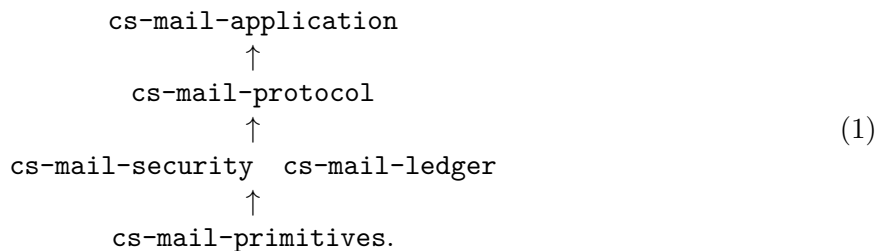
Public errors expose a stable code, safe explanation, retry classification, and scoped correlation reference. They do not contain content, keys, principal evidence, complete state snapshots, or SQL parameters.

6 How the Codebase Is Divided

The runtime path now gives the crate boundaries a reason to exist. The reference workspace separates the stable concerns:

```
crates/
  cs-mail-primitives
  cs-mail-security
  cs-mail-ledger
  cs-mail-protocol
  cs-mail-application
  cs-mail-capabilities
  cs-mail-privacy
  cs-mail-content
  cs-mail-wire
  cs-mail-storage-postgres
  cs-mail-service
  cs-mail-worker
  cs-mail-client
  cs-mail-adapters
```

The dependency direction is



The arrows describe dependency, not data flow. The primitives crate defines constrained money, time, versions, and scoped references. It contains no database, network, asynchronous runtime, or business workflow.

The security crate verifies signatures and signing domains, constructs pairwise identifiers, supports key rotation and transparency, and provides client-side content-encryption building blocks. It turns authenticated bytes into evidence. It does not decide the semantic effect of an acceptance.

The ledger crate defines accounts, reservations, journal batches, postings, and balance projections. It validates conservation and idempotency. It does not decide which allocation acceptance or rejection requires.

The protocol crate defines commands, state, invariants, and the pure transition. It combines security evidence, a complete snapshot, and ledger posting types into a transition manifest. This is the deterministic kernel.

The capabilities crate defines recipient-signed lane grants, native and legacy subjects, rate and lifetime enforcement, and canonical bond-free admission requests. Opaque handles identify native pairs but do not themselves confer authority.

The PostgreSQL storage crate coordinates settlement manifests and bond-free admission under the same relationship aggregate lock. It validates durable encrypted content, consumes lane allowance, advances the canonical journal, and writes delivery effects atomically. Blocking revokes a lane in the same database transaction as the relationship transition.

The service, worker, wire, content, privacy, client, and adapter crates supply authenticated ingress, leased work, endpoint encryption, scoped references, and external boundaries. An adapter may fail because an external system is unavailable; it may not substitute a different protocol meaning.

The binaries assemble these libraries for a deployment. During early development, several roles may run in one binary. Before real user data is accumulated, privacy- sensitive roles need separate credentials, keys, and storage access even when one organization operates all of them.

A concern should become another crate when it has a stable interface and an independent reason to compile, test, reuse, or release. It should become another process when it needs a trust boundary, failure domain, scaling model, or separate operator. These are not the same decision. A crate does not stop its hosting process from reading every database.

7 Privacy and Security Boundaries

The architecture protects message content, relationship state, contact-attempt outcomes, and their financial linkage. Achieving that goal requires both Rust type boundaries and deployed access boundaries.

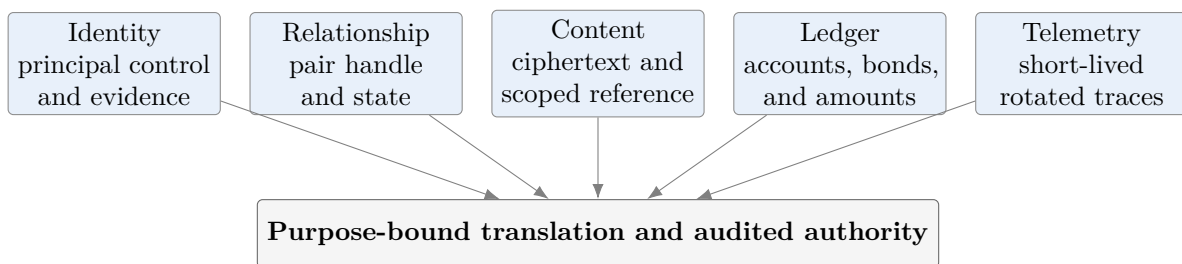


Figure 2: Each privacy domain retains its own references, credentials, and keys. Cross-domain translation is an explicit privileged operation, not a shared global identifier.

7.1 No Routine Component Sees the Whole Story

The identity domain knows that a verified private principal controls a public protocol identity. The relationship domain sees a pairwise handle and current permission. The content domain stores ciphertext under a content-scoped reference. The ledger sees ledger-scoped accounts, bonds, and amounts. Telemetry uses short- lived rotated trace references.

Each operation receives the smallest authority and reference set it needs. A global `UserContext` containing legal identity, every address, ledger account, relationship handles, and tracing identifiers would defeat this design even if every field had a strong Rust type.

The different domains should use separate database credentials, encryption keys, and access roles. Translation among scoped references is a privileged operation performed for a specific purpose and recorded in an audit trail that does not itself reproduce the graph.

7.2 Pairwise Handles Resist Graph Reconstruction

The relationship between I_A and I_B is represented outside the routing and display boundary by an opaque pairwise handle. Alice’s relationship with another recipient uses an unrelated handle, and the reverse relationship from Bob to Alice uses another. A publicly testable hash of two addresses is not opaque because an observer can guess addresses and recompute it.

Repeated-attempt enforcement needs continuity across Alice’s addresses, so the identity domain creates a recipient-scoped assertion for (P_A, I_B) . Bob’s provider can address the correct anti-persistence history without learning a stable principal value that other recipients can compare.

Pairwise handles reduce direct linking but do not eliminate traffic analysis. Precise timestamps, network addresses, message sizes, distinctive bond amounts, and synchronized logs can reconnect events. Retention should be short, tracing identifiers should not cross privacy domains, and batching or aggregation should be used where it does not weaken settlement correctness.

7.3 Native Content Encryption Ends at User Devices

The sender client encrypts the subject, body, inline media, and attachments before upload. The content service stores authenticated ciphertext and a scoped reference. Authorized recipient devices decrypt it. Relationship and ledger events never contain plaintext content or content-derived search terms.

Operational signing keys, message-content keys, pairwise derivation keys, and retention keys serve different purposes and should use separate hierarchies. Key transparency or an equivalent auditable mechanism makes silent operational-key substitution detectable. Account recovery restores authority through an explicit authenticated protocol; it is not a hidden provider key for decrypting old native messages.

An SMTP gateway may necessarily handle provider-readable content. The client and wire envelope must present that route as a privacy downgrade. Transport TLS and provider-managed encryption at rest are not equivalent to native end-to-end encryption.

7.4 Retention Is Executable Policy

Every stored category has a retention class describing purpose, controlling key, operational lifetime, dispute lifetime, separately required retention, and deletion procedure:

```
pub struct RetentionClass {
    pub id: RetentionClassId,
    pub purpose: RetentionPurpose,
    pub operational_until: ExpirationRule,
    pub dispute_until: Option<ExpirationRule>,
    pub required_until: Option<ExpirationRule>,
    pub key_scope: RetentionKeyScope,
    pub deletion: DeletionPlan,
}
```

After settlement and the applicable dispute period, the system removes unnecessary quote detail, delivery telemetry, and cross-domain linkage. A continuing accepted relationship retains the opaque current permission and revocation material it needs, not every event that formed the relationship. A required financial record may remain longer under a separate key and access role.

Cryptographic erasure is complete only when replicas, backups, wrapped-key tables, recovery escrow, and debug exports cannot restore the destroyed linkage. A deletion worker records a nonsecret deletion manifest, and immutable backups lose the relevant keys or expire on a bounded schedule.

A consent-locked archive can exist as a separate encrypted user object whose reconstruction requires sender and recipient shares. Live permission, revocation, backoff, and pending settlement cannot depend on both parties retaining those shares. Any independently decryptable compliance record is a separate retention class and must not be described as two-party-only.

7.5 Logs Are Part of the Privacy Model

Sensitive structures should omit `Debug` or implement a redacted form. Structured logging accepts deliberately safe event types rather than serializing commands or state indiscriminately. Logs contain no plaintext message content, raw keys, private identity evidence, full balances, or reusable global relationship identifier.

Tests should capture logs and error reports and prove those absences. An observability system with every identifier can reconstruct the graph even when the primary databases are carefully separated.

8 Wire Formats and Federation

Wire representations are untrusted input and should not double as internal domain types. A Protobuf, CBOR, or JSON value may be missing, oversized, noncanonical, or associated with an unsupported version. Decoding produces a versioned DTO, not a valid protocol command.

Conversion performs semantic validation:

```
impl TryFrom<wire::V1AcceptRelationship>
  for AcceptRelationship
{
  type Error = ValidationError;

  fn try_from(dto: wire::V1AcceptRelationship)
    -> Result<Self, Self::Error>
  {
    // Validate scope, lengths, versions, and canonical forms.
  }
}
```

Signatures cover one canonical byte representation. Signing-domain tags include protocol version, command type, deployment or federation domain, and intended provider where appropriate. Parsing and reserializing an ambiguous format before verification must not allow signer and verifier to authorize different meanings.

Federation reuses the centralized transition semantics. A sender provider prepares a reservation commitment. A recipient provider verifies scoped assertions and admits the message. Authenticated decisions create settlement obligations, and providers clear them without revealing content or a globally stable private principal.

The centralized manifest is the reference meaning of a transition. A future distributed prepare-and-commit protocol may change how providers reach finality; it must not change which bonds acceptance refunds or where rejection transfers value. Golden vectors allow independent implementations to compare signing bytes, pairwise handle derivations, transition outcomes, and ledger postings.

9 Testing the Architecture

The first tests should tell the same story as the opening section. Given an unknown relationship and three decision-open admitted bonds, acceptance establishes permission and refunds all three. Given one reserved bond, one logically expired admitted bond, and two decision-open admitted bonds,

acceptance fully cancels the first, expires the second, refunds the other two, grants permission, and never reopens the expired bond. Given a duplicate command, the second execution moves no value.

Reservation tests cancel before the admission deadline, time out after it, and race admission against cancellation. Cancellation always returns $C + S + L_k$, creates no solicitation or decision deadline, and leaves repeated-attempt state unchanged. Winning admission makes encrypted content and a delivery intent durable, starts the decision window, advances attempt state, and makes the cancellation job a no-op.

Solicitation tests admit three distinct messages for the same unaccepted directed relationship and require one establishment effect, one active generation, three distinct attempts, and three independently correct bond deadlines. They then accept, reject, block, or expire the complete sequence and require the generation to close exactly once. After rejection, a later eligible attempt may open the next generation. After blocking, terms, reservation, and admission fail until an authenticated unblock; unblocking preserves backoff and opens no episode by itself. An exact replay of any earlier admission creates neither a new attempt nor a new effect.

Identity-scope tests give one principal two public identities. Accepting the first settles only its bonds, fully cancels only its unadmitted reservations, and releases only its active persistence reserves. The second identity remains unaccepted, its reserves remain scheduled, and the shared principal-level attempt state does not reset.

Property-based tests generate sequences of quotes, reservations, cancellations, admissions, decisions, expiries, releases, revocations, blocks, and unblocks. After every transition they assert value conservation, bond finality, reserve finality, complete relationship-wide settlement, and replay safety. Generators emphasize admission and decision boundary timestamps, maximum integer values, zero components where permitted, many pending bonds, and duplicate commands.

The ledger has its own algebraic tests. Every batch balances by settlement unit. No reserve releases twice. Balances rebuilt from the journal equal stored projections.

Concurrency tests run admission against cancellation and run acceptance, rejection, blocking, unblocking, expiry, reservation, and revocation against the same versions in different orders. Exactly one conflicting transaction commits; the other reloads or becomes a no-op. Fault injection crashes after each transactional step and external effect to prove that the journal, outbox, and idempotency rules recover without partial settlement.

Privacy tests show that a `PrincipalRef` cannot be passed where a `LedgerAccountRef` is required, service credentials cannot query another privacy domain, and captured logs contain no prohibited data. Retention tests advance canonical time, run deletion, exercise eligible backup restoration, and prove that expired linkage cannot be reconstructed.

Wire decoders, canonical encoders, signature verification, and encrypted-content handling should be fuzzed. Unsafe Rust should remain absent from protocol and ledger code unless a narrowly reviewed dependency requires it. Memory safety is a foundation; it does not by itself prove authorization, accounting, privacy, or transaction correctness.

10 Implementation Sequence

The first milestone is an in-memory model of the pure kernel and ledger. It accepts explicit state, authenticated commands, canonical time, and fixed policy, then produces manifests, including unadmitted cancellation, identity-scoped acceptance, principal-level history preservation, and the distinct reject, block, and unblock transitions. Scoped capabilities are deliberately absent from the settlement kernel. Story tests and property tests establish the protocol before a database can obscure mistakes.

The second milestone places the same kernel inside one transactional deployment. It adds the append-only ledger, versioned current state, a complete nonterminal- bond index, scheduled commands, and the transactional outbox. Concurrency and crash tests establish that the durable shell preserves kernel semantics.

The third milestone establishes production privacy boundaries before substantial user data accumulates. Identity, relationship, content, ledger, and telemetry use scoped identifiers, separate credentials, separate keys, and executable retention. Native clients add endpoint encryption, key transparency, and recovery.

Express lanes implement scoped capabilities in the durable admission layer once the base admission and relationship semantics are stable. They bypass bonds without opening a solicitation or advancing repeated-attempt state, and a recipient block atomically revokes them. SMTP compatibility follows as an explicit edge adapter. Federation follows after the centralized semantics, privacy boundaries, and settlement invariants are stable. It introduces genuinely independent providers and adversarial failure assumptions rather than microservices that share one administrator credential and database.

11 Conclusion

The architecture is easiest to understand through one acceptance. Bob’s authenticated command and a complete state snapshot enter a pure Rust transition. The transition produces a complete settlement manifest. One transaction commits permission, bonds, identity-scoped persistence-reserve effects, preserved principal-level repeated-attempt state, ledger postings, schedules, and outbox intentions. Idempotent workers communicate the committed result afterward.

The remaining choices support that flow. Strong types keep identities, money, time, and versions distinct. Purpose-specific storage proves completeness and atomicity. Append-only accounting coexists with direct current-state projections. Async Rust performs I/O around the domain rather than becoming the domain.

Privacy follows the same discipline of explicit boundaries. Endpoints own content keys. Services receive scoped references and only the facts required for their roles. Logs do not become a hidden graph. Retention is executable, and expired linkage is actually destroyed. The resulting codebase can remain small and readable while preserving a credible path toward secure, private federation.